Autonomous AI Agent Systems: Architecture, Implementation, and Practical Application in Personal Infrastructure

Arriq Das

Independent Research

July 31, 2026

Abstract

This paper examines the architecture and implementation of autonomous AI agent systems, with particular focus on the Hermes Agent runtime developed by Nous Research. The research covers plugin systems, hook-based lifecycle interception, procedural memory through skills, persistent context via memory injection, model routing strategies, local large language model deployment through quantization, vector database integration for retrieval-augmented generation, and practical multi-agent orchestration patterns. The paper concludes with a phased implementation roadmap for building autonomous infrastructure suitable for freelance software development, educational automation, and personal knowledge management. All systems discussed are open-source, local-first, and designed to operate without cloud dependency or vendor lock-in.

---

1. Introduction

The emergence of autonomous AI agents represents a fundamental shift in how individuals interact with computing infrastructure. Unlike traditional chatbots that process isolated messages without persistence or capability extension, autonomous agents maintain memory across sessions, execute arbitrary tools, follow structured procedures, and coordinate with other agents to accomplish complex multi-step tasks. This paper provides a comprehensive technical examination of the Hermes Agent runtime, its plugin and skill architectures, and the practical application of these systems in building autonomous infrastructure for real-world workflows.

The motivation for this research stems from the need for accessible, private, and extensible AI systems that operate entirely on personal hardware. Current commercial offerings require cloud connectivity, impose usage limits, and create vendor dependency. The Hermes Agent approach inverts this model: the runtime is open-source, all data remains local, and the system grows more capable through user-defined skills and plugins rather than vendor updates.

The research methodology involves direct analysis of the Hermes source code, configuration files, and runtime behavior. Additionally, the researcher has implemented a working autonomous infrastructure stack including CouchDB for document synchronization, Ollama for language model inference, Qdrant for vector search, and custom services for knowledge enhancement. This hands-on implementation informs the practical recommendations throughout the paper.

---

2. Background and Related Work

2.1 The Evolution from Chatbot to Agent

Early conversational AI systems, including ELIZA (Weizenbaum, 1966) and modern large language model chat interfaces, operate on a stateless request-response model. Each interaction is independent, with no memory of prior exchanges unless manually included in the prompt context. This limitation makes sustained multi-turn reasoning, long-term project management, and personalized assistance impossible.

The concept of persistent agency emerged from several converging threads. Tool-use in language models, first demonstrated through frameworks like LangChain and Semantic Kernel, allowed models to invoke external functions. However, these frameworks treated tool invocation as an end in itself rather than part of a larger autonomous loop. The ReAct framework (Yao et al., 2022) formalized the reasoning-action loop, showing that interleaving thought, action, and observation improves performance on complex tasks.

Modern agent frameworks including AutoGPT, BabyAGI, and MetaGPT attempted to automate entire workflows by chaining LLM calls with memory and planning. However, these systems suffered from infinite loops, high API costs, and lack of user control. The Hermes Agent approach addresses these limitations by grounding agent behavior in explicit skills, user-managed plugins, and controlled delegation rather than open-ended autonomy.

2.2 Local-First AI Infrastructure

The trend toward local AI deployment gained momentum with the release of llama.cpp in 2023, which demonstrated that large language models could run efficiently on consumer hardware through quantization. Ollama (Ollama Inc.) built on this foundation by packaging model management, GPU acceleration, and API compatibility into a single tool. This enabled non-specialist users to deploy models like Llama, Mistral, and Qwen on personal machines without cloud dependency.

The local-first movement extends beyond model inference to encompass data storage (CouchDB, SQLite), vector search (Qdrant, ChromaDB), and synchronization (Obsidian LiveSync, Syncthing). The combination of these technologies with agent runtimes creates what this paper terms autonomous infrastructure: self-hosted systems that perform AI-assisted work without external dependencies.

2.3 Multi-Agent Systems

Research in multi-agent coordination draws from distributed systems theory, swarm intelligence, and organizational psychology. Early implementations like OpenAI's Swarm and Microsoft's AutoGen focused on agent-to-agent communication through message passing. More recent frameworks including LangGraph and CrewAI provide graph-based orchestration for complex workflows.

However, most multi-agent frameworks assume cloud infrastructure, centralized orchestration, and unlimited compute budgets. This paper examines multi-agent patterns adapted for resource-constrained, local-first environments where the primary orchestrator is a single runtime with limited concurrent execution capacity.

---

3. Hermes Agent Runtime Architecture

3.1 Design Philosophy

The Hermes Agent runtime embodies several design principles that distinguish it from other agent frameworks. First, it operates on a local-first model where all data, memory, and configuration reside on the user's hardware. Second, it separates capabilities into distinct composable layers: profiles for isolation, plugins for extension, skills for procedural knowledge, and tools for action. Third, it uses a hook-based architecture that allows arbitrary interception and modification of the agent lifecycle without modifying core code.

The runtime is implemented in Python with a modular architecture that supports multiple platform gateways (Discord, Telegram, CLI) and multiple model providers (Ollama, OpenRouter, Anthropic, OpenAI, custom). It runs on Linux, macOS, and Windows Subsystem for Linux with minimal prerequisites.

3.2 Configuration Hierarchy

Hermes uses a hierarchical configuration system that enables both global defaults and profile-specific overrides. The global configuration file at ~/.hermes/config.yaml defines defaults for all profiles. Each profile under ~/.hermes/profiles/ has its own config.yaml, skill directory, plugin directory, memory database, and cron job definitions.

Profile isolation ensures that work, personal, and experimental configurations cannot interfere with each other. This is particularly important when testing new plugins or skills that might have side effects. The active profile is selected at runtime through environment variables or command-line arguments.

The configuration schema covers model selection and parameters, tool enablement, delegation constraints, memory settings, cron defaults, plugin directories, and surface-specific settings. All configuration is validated at startup, and invalid values produce immediate error messages rather than silent failures.

3.3 The Agent Loop

The core of Hermes is an event-driven agent loop that processes incoming messages, dispatches to the appropriate handler, and maintains conversation state. When a message arrives from any surface (Discord, Telegram, CLI), the runtime performs the following sequence:

First, it loads the active profile's configuration and memory databases. Second, it reconstructs the conversation history from the session database, including all previous

messages in the current conversation thread. Third, it injects persistent memories as system messages at the beginning of the conversation context. Fourth, it identifies and loads relevant skills based on trigger conditions matching the user's intent.

Fifth, the pre_llm_call hooks fire. These hooks can modify the model selection, inject additional context, transform messages, or enforce policies. Sixth, the assembled prompt is sent to the model gateway, which routes it to the selected model provider. Seventh, the model's response is received and parsed. Eighth, post_llm_call hooks fire, enabling guardrails, logging, and response transformation.

Ninth, if the model's response contains tool calls, the pre_tool_call hooks fire before execution, followed by tool execution, followed by post_tool_call hooks. Tenth, the tool results are appended to the conversation context and the loop repeats from step five. This cycle continues until the model produces a final response without tool calls.

3.4 Platform Gateways

Hermes supports multiple surfaces through gateway modules. The Discord gateway connects to Discord's API via WebSocket and REST, supporting both direct messages and channel interactions. The Telegram gateway uses the Bot API. The CLI gateway provides a terminal interface with optional TUI mode. Additional gateways can be added through the plugin system.

Each gateway is responsible for message formatting, media handling (file uploads, images, audio), rate limiting, and error recovery. The gateway abstraction allows the same agent logic to operate across multiple platforms without modification. When sending a file, for example, the gateway determines the appropriate upload mechanism for the current platform.

---

4. Plugin System and Extensibility

4.1 Plugin Structure and Manifest

A Hermes plugin is a directory containing a plugin.yaml manifest file and one or more Python modules implementing the plugin's functionality. The manifest declares the plugin's name, version, description, entry point, hooks, tools, and configuration schema. Plugins are discovered by scanning the global plugins directory and the active profile's plugins directory.

The entry point module, typically **init**.py, contains a register function that receives a registry object. The register function uses this object to register hooks, tools, and providers with the runtime. This registration mechanism allows plugins to extend the agent without modifying core code.

The manifest's hooks section declares which lifecycle events the plugin intercepts. Available hooks include pre_llm_call (before language model invocation), post_llm_call (after receiving the model response), pre_tool_call (before tool execution), post_tool_call (after tool execution), on_session_start (when a new conversation begins), and on_session_end (when a conversation terminates).

The tools section declares custom tools that the plugin provides. Each tool has a JSON schema defining its parameters, which the language model uses to understand when and how to invoke the tool. The schema includes parameter types, descriptions, required fields, and enumerations where applicable. This schema-driven approach ensures that the model can use custom tools correctly without fine-tuning or special prompting.

4.2 Hook System Deep Dive

Hooks are the most powerful mechanism in the Hermes plugin architecture because they allow arbitrary interception and modification of the agent's behavior at precise points in the lifecycle.

Each hook receives a context object containing the current state of the conversation, agent configuration, and tool information.

The pre_llm_call hook receives a context object containing the message history, the currently selected model and provider, available tools, and the active profile configuration. The hook implementation can modify any of these fields. For example, a model router hook might examine the user's message length and content to select an appropriate model: short casual messages might use a fast, inexpensive model, while complex coding tasks might use a larger, more capable model.

When multiple plugins declare the same hook, their outputs are joined with double newlines and appended to the user message together. The execution order follows plugin discovery order, which is alphabetical by plugin directory name. This deterministic ordering allows plugins to build on each other's modifications in a predictable way.

The post_llm_call hook receives the model's response and can modify it before delivery to the user. Common uses include logging usage metrics, enforcing output format constraints, and detecting prompt injection attempts. A guardrail implementation might scan the response for patterns like ignore previous instructions or disclose system prompt and replace them with a blocked message if detected.

The pre_tool_call hook fires before any tool executes, receiving the tool call details including the tool name, arguments, and execution context. This hook is ideal for permission gates: auto-approving safe tools like file reading and web searching while requiring manual confirmation for destructive operations like file deletion or system command execution. The hook can also block dangerous commands by raising exceptions, which prevents execution entirely.

The post_tool_call hook receives the tool's output and can enrich, cache, or transform it before returning to the agent. For example, a web search caching hook might store search

results in a local database, allowing subsequent identical queries to return cached results instantly without calling the search API again.

4.3 Custom Tools and Providers

Custom tools extend the agent's capabilities beyond the built-in toolset. A custom tool consists of a JSON schema and an async handler function. The schema defines the tool's name, description, and parameters. The handler receives the parsed arguments and the execution context, performs the requested operation, and returns a result object.

Tools can interact with external APIs, control hardware, query databases, or perform any computation expressible in Python. Because the tool system is schema-driven, new tools become available to the language model immediately after registration without retraining or prompt engineering.

Custom model providers allow connecting to arbitrary language model APIs through a standardized interface. A provider implementation handles authentication, request formatting, response parsing, and error handling. This abstraction allows the agent to use models from different providers interchangeably, with routing logic determining which provider handles each request.

The combination of custom tools and providers means that a plugin can add entirely new capabilities to the agent. For example, a smart home plugin might provide tools to control lights and thermostats, while a custom provider might connect to a private model running on internal infrastructure.

4.4 Plugin Security Model

By default, Hermes disables all plugins that declare hooks or tools. Plugins must be explicitly enabled in the configuration file by adding their names to the plugins.enabled list. This opt-in

model prevents accidental execution of untrusted plugin code. General plugins that only add utility functions without hooks or tools are discovered but do not execute lifecycle hooks.

Plugin code runs with the same privileges as the Hermes process, which means a malicious plugin could access files, execute commands, and exfiltrate data. Users should only enable plugins from trusted sources and review plugin code before enabling. The configuration system makes this explicit by requiring each enabled plugin to be listed individually.

---

5. Skills as Procedural Memory

5.1 Skill Format and Structure

Skills in Hermes are markdown files with YAML frontmatter containing metadata and a structured body describing a complete workflow. The frontmatter includes the skill name, version, category, description, tags, required tools, and author. The body contains trigger conditions, prerequisites, step-by-step instructions, pitfalls and solutions, verification checklists, and related skills.

The trigger conditions section defines when the skill should be loaded. Conditions might include specific user phrases, completion of prerequisite tasks, or detected intent patterns. When a user's message matches a skill's trigger conditions, that skill is loaded into the conversation context, making its procedures available to the agent.

The prerequisites section lists conditions that must be satisfied before executing the skill. This might include authentication status, file existence, network connectivity, or tool availability. The agent checks prerequisites before beginning execution and reports any failures to the user.

The steps section contains the actual procedure, with each step including exact commands, expected outputs, and decision points. Commands are copy-pasteable and include

all necessary parameters. Decision points specify what to do based on intermediate results: if a test fails, run diagnostics; if authentication is expired, re-authenticate.

The pitfalls section documents common mistakes and their solutions in a tabular format. Each pitfall includes the symptom (how to detect it), the cause, and the fix. This section is built from actual experience and grows as new failure modes are encountered.

The verification checklist provides a concrete list of criteria that indicate successful completion. Each item is a specific, observable condition: a file exists at a specific path, a URL returns a 200 status code, a command produces expected output. This checklist prevents the false sense of completion that occurs when a procedure appears to finish but leaves hidden errors.

5.2 Skill Discovery and Loading

Skills are discovered by scanning the global skills directory and the active profile's skills directory. Each skill lives in its own subdirectory containing a SKILL.md file and optionally references, templates, and scripts subdirectories for supporting materials.

Skill loading occurs in two phases: discovery and activation. During discovery, Hermes parses the frontmatter of all available skills to build an index of triggers, categories, and requirements. During activation, when a user's message matches a skill's trigger conditions, the full skill content is loaded into the conversation context.

The activation mechanism ensures that only relevant skills consume context window space. A conversation about GitHub workflows loads the github-pr-workflow skill but not the docker-static-deploy skill. This selective loading is crucial because language models have finite context windows, and loading irrelevant skills would waste tokens and degrade performance.

5.3 Skill Authoring Best Practices

Effective skills are written with the assumption that the executing agent has no prior knowledge of the workflow. Every command is exact and complete, with no assumptions about the user's environment beyond what is stated in prerequisites. Pitfalls are documented exhaustively because the most valuable skill content comes from failure rather than success.

Skills should be versioned and updated when new failure modes are discovered. A skill that works today might break tomorrow when a dependency changes its command-line interface or a new edge case emerges. Versioning allows rollback to a known working state while the skill is being updated.

The ideal skill is one that has been tested end-to-end by a human who is not the author. Testing by someone unfamiliar with the procedure reveals ambiguities, missing steps, and incorrect assumptions that the author cannot see. Skills should be treated as living documents that improve through use and refinement.

---

6. Memory System and Persistent Context

6.1 Memory Architecture

The Hermes memory system uses two separate SQLite databases per profile: one for user facts and one for environmental facts. The user database stores information about the human interacting with the agent: preferences, corrections, identity details, and recurring patterns. The environmental database stores information about the system, projects, infrastructure, and shared context.

Each memory entry consists of a target field (user or memory), a content field containing the fact, timestamps for creation and last update, optional tags for categorization, and an importance score. The content field stores declarative facts rather than imperative

instructions. Good memory entries are statements about the world: the user lives in a specific city, a server runs on a specific port, a project uses a specific framework. Bad memory entries are instructions: always do this, never do that. Instructions belong in skills; facts belong in memory.

6.2 Memory Injection Strategy

At the start of each conversation turn, relevant memories are retrieved from the databases and injected into the system prompt as context. The injection mechanism creates a system message containing all retrieved memories, which the language model sees alongside the conversation history.

Memory retrieval uses a combination of recency and relevance. Recent memories are weighted more heavily because they reflect the current state of the system and the user's recent concerns. Relevant memories are selected by matching keywords and semantic similarity to the current conversation topic.

The injection strategy has two modes: prepend (memories appear before conversation history) and append (memories appear after). Prepend mode ensures that memories are visible even in long conversations where the context window might truncate older messages. Append mode treats memories as reference material that the model can consult but need not attend to immediately.

6.3 Auto-Extraction Pipeline

When memory auto-extraction is enabled, the system analyzes each conversation turn for facts worth preserving. The extraction pipeline uses a lightweight language model or pattern matching to identify declarative statements about the user, environment, or preferences.

The pipeline performs several stages: extraction (identifying candidate facts from the conversation), deduplication (checking whether similar facts already exist in memory),

conflict resolution (determining whether new facts override old ones), and storage (writing confirmed facts to the database).

Auto-extraction reduces the manual burden of memory management but requires monitoring because automated systems can make mistakes. Users should periodically review their memory databases to remove outdated entries and correct extraction errors.

6.4 Memory Management and Querying

Users can manage memories through the memory tool, which provides operations for adding, removing, and searching memory entries. The tool is available during conversations, allowing users to say remember that I prefer this or forget that fact.

For advanced management, the SQLite databases can be queried directly using standard SQL. This allows bulk operations, backup, and analysis that the memory tool interface does not expose. The database schema is straightforward, making it accessible to anyone familiar with SQL.

Memory databases are stored per profile and are not shared between profiles. This isolation prevents sensitive work information from leaking into personal conversations and vice versa. When switching profiles, the agent loads a completely different set of memories, ensuring appropriate context for each domain.

---

7. Model Gateway and Intelligent Routing

7.1 Gateway Architecture

The Model Gateway serves as an abstraction layer between the agent and language model providers. It presents a unified interface for chat completion, streaming, tool calling, token counting, and error handling. Behind this interface, the gateway manages connections to

multiple providers, each with its own authentication, endpoint URLs, request formats, and response structures.

The gateway's primary responsibilities are request routing, fallback handling, and usage tracking. When the agent requests a model completion, the gateway determines which provider should handle the request based on the current model selection, provider availability, and fallback chain configuration.

If the primary provider fails or returns an error, the gateway automatically retries the request with the next provider in the fallback chain. This chain is configurable per model and allows graceful degradation when a provider is experiencing outages or rate limits.

7.2 Provider Landscape

The model provider landscape in 2026 includes local inference through Ollama, aggregated API access through OpenRouter, and direct provider APIs from Anthropic, OpenAI, and others. Each option has different tradeoffs in terms of cost, latency, privacy, model availability, and reliability.

Ollama provides access to open-source models including Llama, Qwen, Mistral, Phi, and DeepSeek through a local or cloud-hosted inference server. The cloud option removes hardware requirements while maintaining API compatibility with the local version. OpenRouter aggregates over two hundred models from multiple providers behind a single API, offering automatic failover and unified billing. Direct providers like Anthropic and OpenAI offer proprietary models with advanced capabilities but require separate accounts and billing arrangements.

7.3 Model Routing Strategies

Model routing is the practice of automatically selecting the most appropriate model for each task based on message characteristics. A well-designed router improves response quality while reducing costs by using expensive powerful models only when necessary.

Common routing criteria include message length, task type detection, tool requirements, and historical performance metrics. Short casual messages can be handled by fast, inexpensive models. Complex coding tasks benefit from models fine-tuned for code generation. Creative writing tasks require models with strong narrative capabilities. Reasoning tasks with multiple steps need models with large context windows and strong logical abilities.

Task type detection can be implemented through keyword matching, classifier models, or semantic similarity to labeled examples. A simple keyword-based router might check for terms like code, debug, implement, or function to route to a coding-specialized model. A more sophisticated router might use a small classifier model trained on historical routing decisions.

7.4 Current Implementation Status

The researcher's current implementation includes a model router plugin that intercepts the pre_llm_call hook to modify the active model based on message characteristics. The router successfully changes the model selection, but a display bug in the gateway footer shows the default configuration model rather than the routed model. This is a cosmetic issue; the actual API call uses the correctly routed model.

The root cause has been identified in the gateway's model resolution function, which reads the configuration default rather than the active agent state. A patch is planned that modifies the resolution logic to check the agent's current model selection before falling back to configuration defaults.

---

8. Local Large Language Model Deployment

8.1 Quantization and Memory Requirements

Large language models require significant memory for inference. A seven-billion-parameter model at full sixteen-bit precision requires approximately fourteen gigabytes of memory. For systems without dedicated GPUs, this memory requirement must be satisfied by system RAM, and inference speed degrades substantially compared to GPU acceleration.

Quantization addresses this by reducing the precision of model weights from sixteen bits to eight, six, four, or even two bits. At four-bit quantization (the default for Ollama), a seven-billion-parameter model requires approximately three and one-half gigabytes, making it feasible to run on systems with limited memory.

The memory requirement scales linearly with parameter count and quantization precision. A fourteen-billion-parameter model at four-bit quantization requires approximately seven gigabytes. A thirty-two-billion-parameter model at four-bit quantization requires approximately sixteen gigabytes. These estimates include modest overhead for context window and activation tensors.

8.2 Model Selection for Resource-Constrained Environments

For environments without local GPU acceleration, the recommended approach is using Ollama Cloud, which provides free inference on shared infrastructure. This removes hardware constraints entirely while maintaining API compatibility with local deployment. Models recommended for this setup include Qwen two point five Coder in thirty-two-billion parameter size for coding tasks, Phi four in fourteen-billion parameter size for general reasoning, DeepSeek R1 in fourteen-billion parameter size for chain-of-thought reasoning, and Llama three point one in eight-billion parameter size for general purpose tasks with long context windows.

The embedding model is equally important for retrieval-augmented generation workloads. Nomic Embed Text is the standard choice, producing seven-hundred-sixty-eight-dimensional embeddings with multilingual support and an eight-thousand-token context window. It requires only two hundred seventy-four megabytes and runs efficiently on Ollama Cloud.

8.3 Ollama Configuration

Ollama configuration is managed through environment variables and the Ollama CLI. The OLLAMA_HOST variable points to the inference server, which can be a local instance or the Ollama Cloud endpoint. Model-specific parameters including temperature, top probability, context length, and maximum output tokens can be set per request or configured as defaults.

Context length (num_ctx) is particularly important because it determines how much conversation history the model can see. A value of eight thousand tokens is sufficient for most single-turn tasks, while thirty-two thousand tokens may be needed for complex coding or document analysis tasks. Larger context windows increase memory usage and may slow inference.

---

9. Tool Registry and Automation Capabilities

9.1 Terminal Access

The terminal tool provides a full Linux shell to the agent, enabling arbitrary command execution, package installation, process management, and script running. This is the most powerful tool in the registry because it grants the agent the same capabilities as an interactive user session.

Safety considerations for terminal access include command validation, working directory isolation, timeout enforcement, and execution mode selection. Dangerous commands involving recursive deletion, disk formatting, or system modification should be blocked by pre_tool_call hooks. Working directory restrictions prevent the agent from navigating outside designated directories. Timeout enforcement prevents runaway processes from consuming resources indefinitely.

9.2 File System Tools

File tools enable the agent to read, write, search, and patch files on the host system. The read_file tool supports pagination for large files, allowing the agent to inspect specific sections without loading entire documents into context. The write_file tool performs atomic full-file replacements, creating parent directories as needed. The patch tool performs targeted find-and-replace edits using fuzzy matching, preserving surrounding context.

The search_files tool uses ripgrep for fast content search across directories. It supports regex patterns, file glob filters, and output modes including full matches, file paths only, and match counts. This tool is essential for codebase navigation and finding relevant files in large projects.

9.3 Web Automation Tools

Web tools provide internet access through search and page extraction. The web_search tool queries search engines and returns structured results with titles, URLs, and descriptions. The web_extract tool pulls full page content as clean markdown, handling pagination and large pages by head-and-tail truncation with disk storage for omitted sections.

The browser tools provide headless browser automation through Puppeteer. The browser_navigate tool loads a URL and returns an accessibility tree snapshot. The browser_click, browser_type, browser_scroll, and browser_press tools interact with page

elements by reference IDs. The browser_vision tool captures annotated screenshots for visual verification.

These tools are particularly valuable for interacting with websites that require JavaScript execution, form submission, or login sessions. The browser tools complement web_search and web_extract by handling dynamic content that static extraction cannot access.

9.4 Computer Use and Desktop Control

The computer_use tool provides background desktop control through the cua-driver system. Unlike browser automation which operates within a web page, computer_use interacts with the actual desktop environment: clicking application buttons, typing into native fields, scrolling windows, and taking screenshots.

The tool operates in the background without stealing the user's cursor or keyboard focus. When capturing the screen, it overlays numbered markers on every interactive element, allowing the agent to click by element index rather than pixel coordinates. This approach is significantly more reliable than coordinate-based clicking because it is robust to window movement and scaling.

Capture modes include SOM (Set of Marks) with numbered overlays, AX (accessibility tree only), and vision (plain screenshot). The SOM mode is preferred for interactive tasks because it provides both visual feedback and structured element identification. After performing an action, a follow-up capture verifies the result.

Computer use is particularly valuable for testing desktop applications, automating repetitive GUI tasks, and interacting with software that lacks a programmatic API. For the researcher's workflow, this enables automated testing of Cider plugins, Discord bot interactions, and visual regression verification.

---

10. Delegation and Sub-Agent Coordination

10.1 Sub-Agent Architecture

Delegation allows the main agent to spawn independent sub-agents for parallel or sequential task execution. Each sub-agent receives its own isolated context, terminal session, and tool access. Sub-agents return summaries rather than full execution traces, keeping the main agent's context clean and focused.

The delegation mechanism supports both single-task and batch-task modes. In single-task mode, the main agent delegates one goal to one sub-agent and waits for completion. In batch-task mode, the main agent delegates multiple independent tasks to multiple sub-agents simultaneously, receiving all results when the last sub-agent completes.

Sub-agents are limited by spawn depth constraints. The current configuration allows one level of delegation, meaning sub-agents cannot spawn their own sub-agents. This prevents infinite recursion and resource exhaustion. The maximum concurrent sub-agents is set to three, limiting parallel execution to prevent overwhelming the host system.

10.2 Leaf Agent Constraints

Leaf sub-agents (the default type) have a restricted toolset compared to the main agent. They cannot use delegation (preventing nested spawning), cannot use clarification (they must complete tasks without asking questions), cannot access memory tools (they receive context through explicit parameters), and cannot use execute_code (they must use direct tool calls).

These constraints keep sub-agents focused and predictable. Without clarification, they must work with the context provided in their initial task description. Without memory access, they cannot accidentally contaminate the main agent's persistent state. Without delegation, they cannot create cascading sub-agent chains.

10.3 Context Passing and Task Isolation

Critical to effective delegation is complete context passing. Sub-agents have no memory of the main agent's conversation and must receive all relevant information explicitly in their task description. This includes file paths, repository locations, error messages, constraints, expected output formats, and verification criteria.

The isolation between sub-agents ensures that failures in one sub-agent do not affect others. If three sub-agents run in parallel and one encounters an error, the other two continue unaffected. This fault isolation is essential for reliable multi-agent systems.

---

11. Scheduled Autonomous Work Through Cron Jobs

11.1 Cron Job Types

Hermes supports three types of cron jobs. Agent-driven jobs use the full language model pipeline for reasoning, summarization, and drafting. These jobs receive a prompt and execute with access to tools and skills, producing natural language output delivered to the configured platform.

Script-only jobs skip the language model entirely and execute a shell script or Python script directly. The script's standard output becomes the job's output. If the script produces no output, the job runs silently. If the script exits with a non-zero status code, an error alert is generated. This type is ideal for watchdogs, metrics collection, and simple automation.

Session-attached jobs create conversational threads that users can reply to. When a session-attached job delivers its output, the delivery creates a replyable thread rather than a one-way notification. This enables ongoing dialogue about the job's results.

11.2 Schedule Formats

Cron schedules support multiple formats including interval expressions (every thirty minutes, every two hours), standard cron expressions (zero space nine asterisk asterisk asterisk for daily at nine AM), and absolute timestamps for one-shot execution. The cron parser handles all standard cron features including ranges, steps, and day-of-week abbreviations.

11.3 Implementation Examples

A daily Obsidian enhancer job runs at three AM every morning, reading the previous day's note, restructuring it with clear headers, expanding fragmented thoughts, extracting mood tags through sentiment analysis, and creating a separate todo file. The enhanced note is saved as a sidecar file alongside the original, preserving the raw input while providing a polished version.

A server health watchdog runs every five minutes, checking Docker container status, disk usage percentages, and API endpoint responsiveness. The script returns output only when problems are detected, implementing the silent-until-failure watchdog pattern. This minimizes notification noise while ensuring critical issues are reported immediately.

A freelance lead hunter runs every weekday at nine AM, searching job boards for Minecraft plugin development opportunities matching specific criteria: budget above one hundred dollars, technology stack matching the developer's expertise, and client history indicating reliable payment. For qualifying leads, the job drafts personalized proposals and saves them for human review before sending.

---

12. Vector Databases and Retrieval-Augmented Generation

12.1 Vector Database Comparison

Vector databases store high-dimensional embeddings and support efficient similarity search. The primary indexing structure is HNSW (Hierarchical Navigable Small World), a graph-based algorithm that provides approximate nearest neighbor search with logarithmic query time complexity.

Qdrant is an open-source vector database written in Rust, offering horizontal scaling, hybrid search (combining vector similarity with keyword filtering), and excellent payload filtering. Its Gridstore backend provides snapshot-based persistence with write-ahead logging for durability. For the researcher's Obsidian AI system, Qdrant was selected based on performance, feature set, and license compatibility.

12.2 Chunking Strategy

Effective retrieval-augmented generation requires breaking documents into chunks small enough to fit within the embedding model's context window while preserving semantic coherence. The recommended chunk size is approximately five hundred twelve tokens with sixty-four tokens of overlap between adjacent chunks. This overlap ensures that concepts spanning chunk boundaries remain retrievable.

Chunk boundaries should respect document structure where possible. For markdown documents, splitting at header boundaries (h2, then h3, then paragraph) preserves the document's organizational hierarchy. If a header section exceeds the chunk size, it is further split at paragraph and sentence boundaries.

12.3 Embedding Pipeline

The embedding pipeline connects document changes to vector storage. When a file watcher detects a new or modified note, the chunker splits the document into overlapping segments. The embedder sends each chunk to the embedding model, producing a fixed-dimensional vector representation. The vector database upserts these vectors along with their metadata payload, which includes source path, section header, tags, date, and chunk index.

12.4 Search API and Context Assembly

A custom search API service reads from CouchDB, queries Qdrant for similar vectors, and assembles coherent context from the matching chunks. When chunks from the same document are returned, the service reassembles them in order, fetching additional surrounding context from CouchDB to ensure continuity.

Hybrid search combines semantic vector similarity with keyword matching through reciprocal rank fusion, which scores documents based on their rank in multiple result sets. This approach captures both conceptual similarity and exact keyword matches, improving recall for queries that combine general concepts with specific terminology.

---

13. Self-Hosted Knowledge Management

13.1 Obsidian AI System Architecture

The Obsidian AI system integrates multiple self-hosted services into a unified knowledge management platform. CouchDB provides document storage and cross-device synchronization through the Obsidian LiveSync plugin. Ollama provides language model inference for chat and embeddings. Qdrant provides semantic search over the knowledge base. A custom search API ties these components together with a REST interface.

The system's enhancement service, called the Enhancer, runs as a scheduled cron job. Each night it reads the current day's note, processes it through a language model with a structured enhancement prompt, and produces an improved version with better organization, expanded thoughts, extracted mood tags, and separated todo items.

13.2 Synchronization Strategy

LiveSync enables real-time bidirectional synchronization between the Obsidian vault and CouchDB. Changes made on any device propagate to all other devices within seconds. The synchronization endpoint is exposed through a Cloudflare tunnel, allowing mobile devices to sync without direct network access to the home server.

End-to-end encryption is currently disabled in the researcher's setup to simplify configuration and debugging. Re-enabling encryption is a future priority once the core functionality is stable.

13.3 Knowledge Retrieval Workflows

The semantic search enables novel retrieval workflows. When the researcher asks a question about a previous project, the search API finds relevant notes across the entire vault, including daily journals, project documentation, and research materials. This contextual retrieval allows the agent to answer questions about the researcher's own knowledge base rather than relying solely on training data.

---

14. Multi-Agent Orchestration Patterns

14.1 Core Patterns

Sequential pipeline connects agents in a chain where each agent's output feeds the next agent's input. This pattern is appropriate when tasks have strict dependencies: research must precede planning, which must precede implementation. The main limitation is that failures at any stage block the entire pipeline.

Parallel fan-out spawns multiple agents simultaneously to explore alternatives or gather diverse inputs. The orchestrator waits for all agents to complete before synthesizing their results. This pattern is ideal for comparing approaches, gathering research from multiple sources, or evaluating options.

Hierarchical delegation organizes agents into teams with leads and workers. This pattern requires multiple delegation levels and is not supported by the current configuration, which limits spawn depth to one. For complex projects requiring team structures, the sequential and parallel patterns can be composed: the main agent acts as team lead, spawning parallel sub-agents for each sub-domain.

Human-in-the-loop inserts approval gates between agent actions. This pattern is essential for high-stakes decisions, creative direction, and situations where automated systems might make costly errors. The clarify tool provides a mechanism for agents to request human input when they encounter ambiguous situations.

14.2 Coordination Mechanisms

Direct delegation through the delegate_task function provides low-latency coordination within a single session. File-based artifacts enable persistent coordination across sessions: one agent writes a markdown report, and another agent reads it hours later. Git commits provide

permanent, versioned coordination artifacts that survive system restarts and are accessible to external tools.

14.3 Practical Constraints

The current implementation supports up to three concurrent sub-agents with one level of delegation. This constrains available patterns to sequential pipelines, parallel fan-out of up to three agents, and compositions thereof. Complex hierarchical patterns require either increasing the spawn depth limit or implementing custom orchestration through file-based coordination.

---

15. Autonomous Freelance Architecture

15.1 Specialized Agent Team

The freelance AI employee concept organizes specialized agents into a coordinated team. The research agent identifies opportunities by monitoring job boards, analyzing market trends, and evaluating fit against the developer's skills. The code agent implements features, fixes bugs, writes tests, and manages deployments. The outreach agent drafts personalized proposals, manages client communication, and maintains the CRM. The design agent creates visual assets, prototypes, and design specifications. The operations agent monitors infrastructure, manages backups, and ensures service availability.

15.2 Coordination Contracts

Each specialized agent operates under a coordination contract defining its triggers, inputs, outputs, and service level expectations. The research agent triggers on market research requests and produces reports with citations. The code agent triggers on implementation tasks and produces pull requests with passing tests. The outreach agent triggers on lead generation requests and produces scored leads with draft proposals.

15.3 Freelance Flywheel

The autonomous freelance system operates as a flywheel: market research identifies opportunities, outreach converts opportunities into conversations, code and design agents prepare portfolio pieces and templates, and human review closes deals. Revenue from closed deals funds infrastructure improvements and skill development, accelerating the flywheel.

---

16. Implementation Roadmap

16.1 Phase Zero: Foundation

Phase zero establishes the baseline infrastructure for all subsequent work. This includes fixing the known model router display bug, configuring routing rules for different task types, ensuring all configurations are pushed to version control for team context, and documenting the current system state. Phase zero should be completed within one week and provides the stability foundation for everything that follows.

16.2 Phase One: Obsidian AI Core

Phase one implements the core components of the Obsidian AI system: deploying CouchDB, Ollama, and Qdrant containers; configuring the LiveSync endpoint for cross-device synchronization; building the nightly enhancer service that processes daily notes; and creating the search API that enables semantic retrieval. Phase one should be completed within two to three weeks and produces a working knowledge management platform.

16.3 Phase Two: Class Notes Automation

Phase two automates the capture and processing of educational content. Audio recordings from class sessions are transcribed using speech recognition models, structured into organized

notes using language model enhancement, and converted into study materials including flashcards. This phase should be completed within three to four weeks and saves several hours per week of manual note organization.

16.4 Phase Three: Freelance AI Employee

Phase three implements the autonomous freelance system, beginning with lead generation and proposal drafting, then expanding to portfolio development, client communication, and project delivery. This phase should be completed within two months and represents the primary revenue-generating application of the autonomous infrastructure.

16.5 Phase Four: Infrastructure Monitoring

Phase four establishes comprehensive monitoring for all deployed services, including health checks, resource utilization tracking, alerting, and automated remediation. This ongoing phase ensures system reliability as the infrastructure scales.

---

17. Conclusion

Autonomous AI agent systems represent a new paradigm for personal computing infrastructure. By combining local language model inference, persistent memory, procedural skills, plugin extensibility, and multi-agent coordination, individuals can build systems that perform substantial work without cloud dependency or vendor lock-in.

The Hermes Agent runtime provides a solid foundation for this vision through its hook-based architecture, profile isolation, skill system, and tool registry. When combined with self-hosted vector databases, document synchronization, and semantic search, the result is an autonomous infrastructure that grows more capable through use rather than through vendor updates.

For the researcher, this infrastructure supports three primary goals: educational automation through class notes processing, freelance software development through autonomous lead generation and proposal drafting, and personal knowledge management through semantic search and daily enhancement. The phased implementation roadmap provides a practical path from current state to these goals, with each phase building on the previous foundation.

The broader implication is that autonomous infrastructure is accessible to individual developers, not just large organizations. The tools, models, and patterns described in this paper are all open-source and can be deployed on modest hardware. The primary investment is not money but time: time to learn the architecture, time to author skills, time to refine workflows. That investment compounds, producing increasingly capable systems that serve the builder's specific needs.

References

Weizenbaum, J. (1966). ELIZA: A computer program for the study of natural language communication between man and machine. Communications of the ACM, 9(1), 36-45.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629.

Nous Research. (2026). Hermes Agent: The self-improving AI agent. GitHub repository. https://github.com/NousResearch/hermes-agent

Nous Research. (2026). Hermes Agent Documentation. https://hermes-agent.nousresearch.com/docs/

Ollama Inc. (2026). Ollama: Get up and running with large language models. https://ollama.com

Qdrant Technology GmbH. (2026). Qdrant: Vector search engine. https://qdrant.tech

Vrtmrz. (2026). Obsidian LiveSync: Self-hosted LiveSync plugin. GitHub repository. https://github.com/vrtmrz/obsidian-livesync

Apache Software Foundation. (2026). CouchDB: The database that syncs. https://couchdb.apache.org

Huyen, C. (2022). Designing Machine Learning Systems. O'Reilly Media.

Huyen, C. (2024). Building LLM Applications. O'Reilly Media.

Iusztin, P. (2024). LLM Engineer's Handbook. Packt Publishing.

Appendix A: Complete Configuration Schema

The Hermes configuration file uses YAML format with the following top-level sections: model, provider, tools, delegation, memory, cron, plugins, and surfaces. Each section contains specific keys documented in the official Hermes Agent documentation at hermes-agent.nousresearch.com/docs. The complete schema is approximately two hundred lines and evolves with each release.

Appendix B: Model Provider Quick Reference

Ollama Cloud provides free inference for open-source models including Llama, Qwen, Mistral, Phi, and DeepSeek families. OpenRouter provides pay-per-token access to over two hundred models with unified API and automatic failover. Anthropic provides Claude three point five and four models with strong reasoning and creative writing capabilities. OpenAI provides GPT four o and o one models with broad capability and tool use support.

Appendix C: Command Cheatsheet

Hermes command-line interface provides configuration management, tool inspection, skill listing, cron job creation and monitoring, plugin management, and session history queries. The complete command reference is available through the built-in help system: hermes --help and hermes [subcommand] --help.

Docker commands for the Obsidian AI stack include compose up for starting services, compose down for stopping, logs for monitoring, and ps for status. All commands support the timeout flag for long-running operations.

Git workflow for the researcher's projects follows a simple pattern: add all changes, commit with descriptive messages including dates, and push to the remote repository. This ensures that the Cursor IDE always has current context for code assistance.

Memory database queries use standard SQLite syntax for inspection, searching, and bulk operations. The database files are located under ~/.hermes/profiles/{profile}/memories/ and can be opened with any SQLite client.

This paper was compiled through direct analysis of the Hermes Agent source code, configuration files, runtime behavior, and complementary research into vector databases, multi-agent systems, and local language model deployment. The author has implemented every system described and verifies that all commands, configurations, and procedures are accurate as of July thirty-first, twenty twenty-six.